Kubernetes and Cloud Native Associate - KCNA

Container Orchestration Security

KubeConfig

Welcome to this comprehensive lesson on kubeconfig in Kubernetes. In this guide, we will discuss how kubeconfig files streamline authentication and context management when interacting with Kubernetes clusters. By consolidating configuration details into a single file, you can avoid the repetitive input of authentication parameters for each command.

Overview

Kubeconfig files simplify cluster access by encapsulating details such as the API server address, client certificates, keys, and supported contexts. Initially, you might have generated a certificate for a user and used `curl` to query the Kubernetes REST API. For instance, in a cluster named "my kube playground," a typical curl request looks like this:

```
curl https://my-kube-playground:6443/api/v1/pods \
  --key admin.key \
  --cert admin.crt \
  --cacert ca.crt
```

The API server authenticates this request using the certificate data. In contrast, the `kubectl` command can use command-line options to specify the server, client key,

client certificate, and certificate authority. However, inputting these options every time is tedious. Instead, you can consolidate these details into a kubeconfig file.

Using Kubeconfig with kubectl

Rather than invoking:

```
kubectl get pods \
  --server my-kube-playground:6443 \
  --client-key admin.key \
  --client-certificate admin.crt \
  --certificate-authority ca.crt
```

you can create a kubeconfig file. When you place this file at **`~/.kube/config`**, kubectl will use it by default:

```
kubectl get pods --kubeconfig config
```

which might output:

```
No resources found.
```

Note

Once your kubeconfig file is saved in **`~/.kube`**, there is no need to repeatedly specify the file location or include the authentication details with every kubectl command.

Structure of a Kubeconfig File

A kubeconfig file in YAML format is organized into three primary sections:

- **Clusters:** Represent various Kubernetes clusters (e.g., development, testing, production, or cloud-based clusters).
- **Users:** Contain credentials and client certificate information for users (e.g., admin, dev, prod).
- **Contexts:** Link clusters and users to define which user accesses which cluster. For example, a context named "admin@production" may indicate that the admin account is used to access the production cluster.

Below is an example of a kubeconfig file demonstrating this structure:

```
apiVersion: v1
kind: Config
clusters:
- name: my-kube-playground
  cluster:
    certificate-authority: ca.crt
    server: https://my-kube-playground:6443
contexts:
- name: my-kube-admin@my-kube-playground
  context:
    cluster: my-kube-playground
    user: my-kube-admin
users:
- name: my-kube-admin
  user:
    client-certificate: admin.crt
    client-key: admin.key
```

You can specify the default context by adding the `current-context` field. For example:

```
apiVersion: v1
kind: Config
current-context: dev-user@google
clusters:
```

```
- name: my-kube-playground  # values hidden…
- name: development
- name: production
- name: google
contexts:
- name: my-kube-admin@my-kube-playground
- name: dev-user@google
- name: prod-user@production
users:
- name: my-kube-admin
- name: admin
- name: dev-user
- name: prod-user
```

To view your kubeconfig settings, run:

```
kubectl config view
```

If you do not specify a kubeconfig file, kubectl defaults to the file located at **`~/.kube/config`**. Alternatively, you can explicitly specify a kubeconfig file via the command line:

```
kubectl get pods --kubeconfig my_custom_config
```

Below is another example of a default kubeconfig file that could be moved to the home directory:

```
apiVersion: v1
kind: Config
current-context: kubernetes-admin@kubernetes
clusters:
- cluster:
    certificate-authority-data: REDACTED
    server: https://172.17.0.5:6443
  name: kubernetes
contexts:
```

```
- context:
    cluster: kubernetes
    user: kubernetes-admin
  name: kubernetes-admin@kubernetes
users:
- name: kubernetes-admin
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
```

And here is a simplified version:

```
apiVersion: v1
kind: Config
current-context: my-kube-admin@my-kube-playground
clusters:
- name: my-kube-playground
- name: development
- name: production
contexts:
- name: my-kube-admin@my-kube-playground
- name: prod-user@production
users:
- name: my-kube-admin
- name: prod-user
```

Updating the Current Context

To update your current context, use the following command. For instance, to switch from "my-kube-admin@my-kube-playground" to "prod-user@production", run:

```
kubectl config use-context prod-user@production
```

After executing the command, your kubeconfig file updates to a state similar to:

```
apiVersion: v1
kind: Config
current-context: prod-user@production
clusters:
- name: my-kube-playground
- name: development
- name: production
contexts:
- name: my-kube-admin@my-kube-playground
- name: prod-user@production
users:
- name: my-kube-admin
- name: prod-user
```

You can further manage and view your kubeconfig file using other variations of the `kubectl config` commands. For example, to view the current configuration:

```
kubectl config view
```

Working with Namespaces

Kubernetes clusters often incorporate multiple namespaces. You can specify a namespace in your kubeconfig file so that switching contexts automatically sets the active namespace. Here's an example:

```
apiVersion: v1
kind: Config
clusters:
- name: production
  cluster:
    certificate-authority: ca.crt
    server: https://172.17.0.51:6443
contexts:
- name: admin@production
  context:
    cluster: production
```

```
    user: admin
    namespace: finance
users:
- name: admin
  user:
    client-certificate: admin.crt
    client-key: admin.key
```

Certificates in Kubeconfig

By default, kubeconfig files reference file paths for certificates (for example, `` `certificate-authority: ca.crt` ``). Often, it is preferable to provide full paths or embed the certificate data directly in the file. To embed the certificate, encode it using base64 and specify it under the field `` `certificate-authority-data` ``. For example:

```
apiVersion: v1
kind: Config
clusters:
- name: production
  cluster:
    certificate-authority: /etc/kubernetes/pki/ca.crt
    certificate-authority-data: LS0tC...0V1lQjkFReR...
```

The `` `certificate-authority-data` `` entry contains the base64-encoded content of the CA certificate, removing dependency on external file paths. This approach can also be applied to user client certificates and keys.

Security Warning

Always secure your certificate data and avoid exposing sensitive keys or certificate contents in public repositories.

Summary Table

Below is a table summarizing key components of a kubeconfig file and their use cases:

Section	Purpose	Example Entry
Clusters	Specifies the Kubernetes cluster details	`server: https://my-kube-playground:6443`
Users	Contains credentials and certificate info for a user	`client-certificate: admin.crt`
Contexts	Maps a user to a cluster and optionally a namespace	`context: { cluster: production, user: admin, namespace: finance }`
Current Context	Defines the default context for kubectl commands	`current-context: prod-user@production`

Conclusion

Kubeconfig files are a powerful tool for simplifying Kubernetes cluster management. By consolidating the cluster details, user credentials, and context into one file, you can streamline your workflow and enhance your productivity. For more detailed information on managing Kubernetes clusters, consider exploring these resources:

- **Kubernetes Basics**
- **Kubernetes Documentation**
- **Docker Hub**
- **Terraform Registry**

This concludes our lesson on kubeconfig files. Leveraging kubeconfig not only simplifies cluster interactions but also enhances consistent management across multiple environments. Happy clustering!

Watch Video

Watch video content

Previous

TLS in Kubernetes Certificate Creation

Next

API Groups